

# Fast Probabilistic Algorithms for Proving Pairings

Shramee Srivastav  
MIST.cash—FOCBB, Shhtarknet  
Email: shramee@proton.me

September 29, 2025

## Abstract

In this paper, we introduce a public-coin interactive proof system for efficiently proving the correctness of elliptic curve pairing computations. Elliptic curve pairings form the mathematical foundation of many modern cryptographic protocols, yet their verification remains computationally intensive.

In resource-constrained environments—including BitcoinVM, blockchain smart contracts, ZKVMs, and recursive zero-knowledge proof systems—traditional pairing verification remains expensive. This limits the applicability and affordability of privacy-preserving applications, effectively hindering widespread adoption of blockchain technology.

We systematically present a progression of state-of-the-art techniques: El Housni’s foundational R1CS pairing optimizations [Hou22], Feltroidprime’s enhanced extension field arithmetic [Fel23], and Novakovic and Eagen’s final exponentiation elimination [NE24].

Building upon these foundations, our approach extends the Schwartz-Zippel verification framework by consolidating entire Miller loop bit operations into unified polynomial verifications. This achieves a 50% improvement in Miller loop performance. While our method requires additional auxiliary data compared to traditional pairing verification, it achieves a 75% reduction in commitment overhead relative to Feltroidprime’s extension field techniques.

## Contents

|          |                                               |          |
|----------|-----------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                           | <b>3</b> |
| 1.1      | Motivation . . . . .                          | 3        |
| 1.2      | Our Contribution . . . . .                    | 3        |
| 1.3      | Paper organization . . . . .                  | 3        |
| <b>2</b> | <b>Background</b>                             | <b>4</b> |
| 2.1      | Pairing Computation . . . . .                 | 4        |
| 2.2      | Schwartz–Zippel lemma . . . . .               | 5        |
| 2.3      | Constraint-Based Computation Models . . . . . | 5        |

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>3</b> | <b>Related Work</b>                                         | <b>5</b>  |
| 3.1      | Pairings in R1CS – Housni 2022                              | 6         |
| 3.1.1    | Cost of inversions                                          | 6         |
| 3.1.2    | Affine coordinates                                          | 6         |
| 3.1.3    | Avoid doubling                                              | 6         |
| 3.1.4    | Line evaluations                                            | 6         |
| 3.1.5    | Performance                                                 | 6         |
| 3.2      | Faster Extension Field Multiplications – Feltroidprime 2023 | 7         |
| 3.2.1    | Isomorphisms among towered to direct extensions             | 7         |
| 3.2.2    | Multiplication and Schwartz–Zippel lemma                    | 7         |
| 3.2.3    | Deferred Evaluation and Batching                            | 7         |
| 3.2.4    | Performance                                                 | 8         |
| 3.3      | Eliminating Final Exponentiation – Novakovic & Eagen 2024   | 8         |
| 3.3.1    | Equivalence class                                           | 8         |
| 3.3.2    | Equivalence via Residue Check                               | 9         |
| 3.3.3    | Hasse bound and exponentiation by $r$                       | 9         |
| 3.3.4    | Parameters for BN254 curve                                  | 9         |
| 3.3.5    | Performance                                                 | 9         |
| <b>4</b> | <b>Pairing Operations as Polynomials</b>                    | <b>10</b> |
| 4.1      | Strategic Polynomial Segmentation                           | 10        |
| 4.1.1    | Squaring operations and polynomial degree                   | 10        |
| 4.1.2    | Natural segmentation points                                 | 10        |
| 4.2      | Unified Polynomial Framework                                | 12        |
| 4.3      | Comparative Analysis                                        | 12        |
| 4.3.1    | Previous scheme [Fel23]                                     | 12        |
| 4.3.2    | Our scheme                                                  | 13        |
| 4.4      | Bit-Type Polynomials                                        | 13        |
| 4.4.1    | Zero Bit Equation                                           | 13        |
| 4.4.2    | Non-Zero Bit Equation                                       | 13        |
| 4.4.3    | Correction Step Equation                                    | 14        |
| <b>5</b> | <b>Interactive Oracle Proof</b>                             | <b>14</b> |
| 5.1      | Polynomial Accumulation and implementation                  | 14        |
| 5.1.1    | Prover                                                      | 14        |
| 5.1.2    | Verifier                                                    | 15        |
| 5.2      | Performance                                                 | 15        |
| 5.3      | [wip] Protocol Overview                                     | 15        |
| 5.4      | [wip] Single-Round Protocol                                 | 15        |
| 5.4.1    | [wip] Protocol description (Algorithm)                      | 15        |
| 5.4.2    | [wip] Complexity analysis                                   | 15        |
| 5.5      | Two-Round Protocol with RLC Batching                        | 15        |
| 5.5.1    | [wip] Protocol description (Algorithm)                      | 15        |
| 5.5.2    | [wip] Complexity analysis                                   | 15        |
| 5.5.3    | [wip] Comparison with single-round                          | 15        |
| 5.6      | Three-Round Protocol with Native Field Optimization         | 15        |
| 5.6.1    | [wip] Protocol description (Algorithm)                      | 15        |
| 5.6.2    | [wip] Complexity analysis                                   | 15        |

# 1 Introduction

Blockchain technology is ready for widespread adoption—we have reached a scale where processing Visa-like volumes on-chain is within reach. However, privacy remains a significant barrier to entry for many users and applications. Zero-knowledge proofs (ZKPs) have emerged as a powerful solution to this challenge.

## 1.1 Motivation

Elliptic curve pairings represent fundamental building blocks for numerous cryptographic systems including Groth16 [Gro16], PLONK [GWC19], , BLS signature schemes [BGW<sup>+</sup>22], and KZG polynomial commitment schemes [KZG10].

However, **pairings are computationally intensive**. In execution environments such as blockchain virtual machines, recursive proof systems, and arithmetized circuits, the costs associated with this computation become prohibitive for practical deployment.

## 1.2 Our Contribution

We present a novel optimization for Miller loop computation through a public-coin interactive proof system that verifies individual steps involved in the pairing computation as a polynomial under the Schwartz Zippel Lemma for polynomial identity testing. Over the previous technique employing similar design [Fel23] our approach achieves:

- **50% reduction** in the pairing computation complexity
- **75% decrease** in auxiliary data overhead
- Practical deployment enabling ZK proof verification within blockchain transaction limits

Our key insight is that instead of optimizing individual field operations, we can treat complete Miller loop iterations as single polynomial relationships, dramatically reducing both computational and communication costs.

## 1.3 Paper organization

Section 2 provides necessary background on pairings and constraint systems. Sections 3-5 survey the evolution of optimization techniques that form our foundation. Section 6 presents our unified Miller loop optimization framework. Section 7 demonstrates experimental validation and performance analysis. Section 8 concludes with implications and future work.

---

**Algorithm 1** Optimal Ate pairing over Barreto–Naehrig curves [BDM<sup>+</sup>10]

---

**Input:**  $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$

**Output:**  $a_{\text{opt}}(Q, P)$

```
1: Write  $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$  where  $s_i \in \{-1, 0, 1\}$  and  $s_L = 1$ 
2:  $T \leftarrow Q, f \leftarrow 1$ 
3: for  $i = L - 2$  to  $0$  do
4:    $f \leftarrow f^2 \cdot l_{T,T}(P)$ 
5:    $T \leftarrow [2]T$ 
6:   if  $s_i = 1$  then
7:      $f \leftarrow f \cdot l_{T,Q}(P)$ 
8:      $T \leftarrow T + Q$ 
9:   end if
10:  if  $s_i = -1$  then
11:     $f \leftarrow f \cdot l_{T,-Q}(P)$ 
12:     $T \leftarrow T - Q$ 
13:  end if
14: end for
15:  $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$ 
16:  $f \leftarrow f \cdot l_{T,Q_1}(P), T \leftarrow T + Q_1$ 
17:  $f \leftarrow f \cdot l_{T,-Q_2}(P), T \leftarrow T - Q_2$ 
18:  $f \leftarrow f^{p^{12}-1/r}$ 
19: return  $f$ 
```

---

## 2 Background

### 2.1 Pairing Computation

An elliptic curve pairing is a non-degenerate bilinear map  $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ , where  $\mathbb{G}_1$  and  $\mathbb{G}_2$  are groups of points on elliptic curves over finite fields, and  $\mathbb{G}_T$  is a multiplicative group in an extension field. For practical cryptographic applications, we require that this map be efficiently computable.

The computation of an optimal ate pairing—the focus of our implementation—consists of two primary phases:

1. **Miller loop:** The core iterative computation that evaluates line functions at pairs of points from  $\mathbb{G}_1$  and  $\mathbb{G}_2$ , accumulating intermediate results to produce a value in  $\mathbb{F}_{q^{12}}$ .
2. **Final Exponentiation:** This step raises the Miller loop output to the power  $(q^{12} - 1)/r$ , where  $r$  is the order of the groups, ensuring the result lies in the correct subgroup  $\mathbb{G}_T$ .

While both phases present optimization opportunities, the constraint-based verification of these computations introduces unique challenges and possibilities that differ significantly from native implementations.

## 2.2 Schwartz–Zippel lemma

The Schwartz–Zippel lemma (or more precisely, the Demillo–Lipton–Schwartz–Zippel lemma [DL78, Sch79, Zip79, Sch80]) provides a probabilistic method for testing polynomial identities. The lemma states that for a non-zero polynomial  $P(x_1, x_2, \dots, x_n)$  of total degree  $d$  over a finite field  $\mathbb{F}$ , when evaluated at uniformly random field elements, the probability that  $P$  evaluates to zero is at most  $d/|\mathbb{F}|$ .

This probabilistic bound becomes negligible for cryptographically sized fields where  $d \ll |\mathbb{F}|$ , making it practical to verify polynomial identities by evaluation instead of coefficient-wise operations.

## 2.3 Constraint-Based Computation Models

In the context of arithmetization protocols and virtual machines (such as Starkware’s Cairo VM [Sta22], R1CS [GGPR13], Plonkish [GWC19], etc.), where field operations are expressed as algebraic constraints, the computational landscape differs dramatically from native implementations. Operations that are traditionally expensive can become remarkably efficient. Field inversions, for example, can be computed at a fraction of their usual cost, often reducing their relative costs from over  $100\times$  to about the same cost as multiplication. Conversely, seemingly straightforward operations may require complex constraint representations.

Recognizing and exploiting these unique computational properties is essential for unlocking hidden performance potential. Youssef El Housni’s pioneering work **Pairings in Rank-1 Constraint Systems** [Hou22] represents, to our knowledge, the first systematic exploration of these optimization opportunities, providing a breakthrough in the practicality of recursive proving by thoughtfully leveraging the distinctive cost model inherent to constraint-based systems.

## 3 Related Work

The landscape of efficient pairing verification in constraint systems has evolved through several key contributions. El Housni’s **Pairings in Rank-1 Constraint Systems** [Hou22] established the theoretical framework for optimizing pairings in Rank-1 Constraint Systems, introducing techniques for efficient Miller loop implementation and final exponentiation that exploit the unique properties of constraint-based computation models. This work demonstrated that operations traditionally considered expensive in native implementations could be restructured to achieve remarkable efficiency gains in arithmetized environments.

Feltroidprime’s subsequent research on **enhanced extension field multiplication techniques** [Fel23] introduced polynomial verification methods using the Schwartz–Zippel lemma for  $\mathbb{F}_{q^{12}}$  arithmetic operations. By representing extension field elements as polynomials and verifying their operations through polynomial arithmetic, this approach achieved significant performance improvements in the pairing computations, particularly for emulated pairing circuits where traditional field operations are costly.

Novakovic and Eagen’s work **On Proving Pairings** [NE24] presented breakthrough optimization strategies that enable the elimination of final exponentiation overhead through residue witness techniques. Their approach demonstrates that two elements

in  $\mathbb{F}_{q^{12}}$  can be proven equivalent without expensive exponentiation by introducing witness elements that satisfy specific algebraic relationships, effectively embedding the final exponentiation verification into the Miller loop computation itself.

### 3.1 Pairings in R1CS – Housni 2022

El Housni’s seminal work [Hou22] represents the first systematic exploration of pairing optimization within constraint-based computation models, providing techniques that extend beyond R1CS to benefit any arithmetized environment, including Starkware’s Cairo VM [Sta22].

#### 3.1.1 Cost of inversions

Housni notes that the cost of inversions is much lower in constraint systems, almost the same as multiplication. Over  $\mathbb{F}_p$ , inversions cost 2 constraints: one for computing  $x \cdot y$  where  $y$  is provided from the hint, and one equality check  $x \cdot y \stackrel{?}{=} 1$ . Division can be similarly reduced to just two constraints. For  $c = a/b$  where  $c$  is the input from hint, we compute  $b \cdot c$  and equality check  $b \cdot c \stackrel{?}{=} a$ . This is further generalised to field extensions.

#### 3.1.2 Affine coordinates

Since division cost about as much as multiplications, we can represent points in affine coordinates for lower operation counts. Point doubling and addition formulas are provided in section 6.1 of [Hou22]. Doubling costs 12C and addition costs 10C.

#### 3.1.3 Avoid doubling

For double and add operations on non zero bits of the Miller loop, instead of computing  $[2]S + Q$ , computing  $S + Q + S$  saves 2C per non zero bit. Taking this a step further, computation of  $y$  coordinate for the intermediate  $S + Q$  point can be omitted and  $\lambda_1$  can be directly used in computation of  $\lambda_2$ . This **costs 17C**, that’s a **23% savings** over 22C for  $[2]S + Q$ .

#### 3.1.4 Line evaluations

Following [ABLR14], the lines are sparse elements in  $\mathbb{F}_{q^{12}}$  of the form  $ay_P + bx_P \cdot w + c \cdot w^3$  with  $a, b, c \in \mathbb{F}_{p^2}$ . The multiplication of these sparse elements costs 39C. Housni proposes representing the lines in the form  $1 + \frac{b'x_P}{y_P} \cdot w + \frac{c'}{y_P} \cdot w^3$  which makes it sparser (notice unit first coefficient resulting in free multiplication).

Terms  $\frac{x_P}{y_P}$  and  $\frac{1}{y_P}$  can be precomputed at the cost of 2C, and reused throughout the miller loop. Untwisting isomorphisms for this form are given on pages 15–16 in section 6.1 of [Hou22]. This representation only **costs 30C**, that’s a **30% savings** over 39C for the original representation.

#### 3.1.5 Performance

Table 1 summarizes the arithmetic costs for  $\mathbb{F}_{q^{12}}$  as presented in Housni’s paper.

Table 1: Arithmetic in Housni’s paper [Hou22]

|                       | Mul | Square | Div | Sparse Mul |
|-----------------------|-----|--------|-----|------------|
| $\mathbb{F}_{q^{12}}$ | 54C | 36C    | 66C | 30C        |

## 3.2 Faster Extension Field Multiplications – Feltroidprime 2023

Feltroidprime [Fel23] showcases a novel approach to extension field multiplications that exploits a counterintuitive property of circuit-based computation: operations traditionally viewed as expensive (such as polynomial divisions) can become highly efficient in constraint systems. Moving away from the conventional towered extension approach, he proposes returning to direct extensions of  $\mathbb{F}_p$  to achieve significant performance improvements.

### 3.2.1 Isomorphisms among towered to direct extensions

Feltroidprime walks through the construction of mapping functions to go back and forth between direct and towered extensions.

He starts with the tower for BN254 (as used in gnark [Con]) and obtains irreducible polynomial  $-P_{12}(x) = x^{12} - 18x^6 + 82$

And prepares isomorphisms between tower and direct representations [Fel] which we will not include in this paper for brevity.

### 3.2.2 Multiplication and Schwartz–Zippel lemma

Feltroidprime proposes using Euclidean division of polynomials to multiply  $A, B \in \mathbb{F}_{q^{12}}$ . The computation happens as follows:

1. Treat  $A, B$  as polynomials of degree less than 11, with coefficients in  $\mathbb{F}_p$ .
2. Compute the product  $C = A \cdot B$  as a polynomial of degree less than 22 for  $\mathbb{F}_{q^{12}}$ .
3. Perform Euclidean division of  $C$  by the irreducible polynomial  $P_{12}(x)$  to obtain the quotient  $Q$  and remainder  $R$ :  $A(x) \cdot B(x) = Q(x) \cdot P_{12}(x) + R(x)$
4.  $R \in \mathbb{F}_{q^{12}}$  is used as the result of the multiplication between  $A$  and  $B$ .

Inside the circuit, this computation can be provided as a hint and verified with the Schwartz–Zippel lemma. The Fiat–Shamir heuristic can be used to generate the challenge with the transcript.

### 3.2.3 Deferred Evaluation and Batching

In Section 4, Feltroidprime discusses deferring the evaluation to the end of the algorithm and combining it with a polynomial commitment scheme to batch the verifications. This batching transforms hundreds of individual verifications into a single equation. So they can be checked at a single random point, amortizing the costs for Fiat-Shamir hashing.

The batching process is a random linear combination of all the equations to be verified. For  $n$  multiplications, we have:

$$\sum_{i=0}^{n-1} c_i * A_i(z) * B_i(z) = \sum_{i=0}^{n-1} c_i * R_i(x) + \sum_{i=0}^{n-1} c_i * Q_i(z) * P_{12}(z) \quad (1)$$

### 3.2.4 Performance

For performance analysis, we will use the batched verification version.

The implementation receives the computed RHS polynomial aggregation  $Q_{Agg} = \sum_{i=0}^{n-1} c_i * Q_i$  as hint. The LHS aggregation,  $\sum_{i=0}^{n-1} c_i * A_i(z) * B_i(z) - R_i(x)$  is computed within the verifier circuit. While the amortized cost computing the  $Q_{Agg}(z)$  and  $P_{12}(z)$  once in the circuit approaches zero as the number of batched operations increases, we conservatively account for 1C per multiplication to provide fair cost comparison.

**Multiplication costs** 12C each for  $A$ ,  $B$  and  $R$  evaluation, 1C for multiplying  $A(z)$  and  $B(z)$  evaluations, plus 1C for preparing the RLC  $c_i$  multiplication and conservative 1C for RHS aggregation.

For **squarings**, the evaluation of  $A(z)$  is just used again, costing 12C less than multiplication.

**Division** costs multiplication plus additional 12C for assertion.

**Sparse element multiplication** costs Evaluations for  $A$ ,  $B$  and  $R$  where  $B$  contains 5 non-zero coefficients, one out of which is unitary free multiplication. The costs comes at 31C which is 4C more than squaring which has free second polynomial because  $A = B$  for squarings.

Table 2: Arithmetic in Feltroidprime’s paper [Fel23]

|                                                | Mul  | Square | Div  | Sparse Mul |
|------------------------------------------------|------|--------|------|------------|
| $\mathbb{F}_{q^{12}}$                          | 39C* | 27C*   | 51C* | 31C*       |
| *Costs exclude Fiat-Shamir commitment overhead |      |        |      |            |

## 3.3 Eliminating Final Exponentiation – Novakovic & Eagen 2024

The mathematical structure underlying pairing based cryptography involves equivalence classes. Tate pairings (as described in [NE24] and [Cos19]) result in the output in the quotient group  $F_{q^k}/(F_{q^k})^r$ . This requires the output of the Miller loop to be raised to a power of  $(q^k - 1)/r$ . This step, called **the final exponentiation**, normalizes the Miller loop output by clearing the  $r$ -th residue part. This normalization ensures equivalent pairing outputs produce identical results.

This operation is particularly expensive in extension fields like  $\mathbb{F}_{q^{12}}$ , because  $(q^{12} - 1)/r$  becomes very large.

### 3.3.1 Equivalence class

Novakovic and Eagen [NE24] highlight that the final exponentiation can be eliminated when proving pairings. The expensive exponentiation can be replaced with an efficient residue check. The key insight is that instead of normalizing pairing outputs through final exponentiation, we can directly verify equivalence using a residue-witness based approach.

**Equivalence class representation:** Two elements  $A, B \in \mathbb{F}_{q^{12}}$  are equivalent in the pairing context if there exists some witness  $c$  such that  $A \cdot c^r = B$ .

$$A \equiv B \iff \exists c \in \mathbb{F}_{q^{12}} : A \cdot c^r = B$$



### 3.3.2 Equivalence via Residue Check

So we have,

$$A \cdot c^r \equiv A$$

After final exponentiation,

$$(A \cdot c^r)^{(q^{12}-1)/r} \equiv A^{(q^k-1)/r}$$

$$A^{(q^k-1)/r} \cdot c^{r*(q^k-1)/r} \equiv A^{(q^k-1)/r}$$

$$A^{(q^k-1)/r} \cdot c^{q^k-1} \equiv A^{(q^k-1)/r}$$

With Fermat's Little Theorem,

$$A^{(q^k-1)/r} \cdot 1 \equiv A^{(q^k-1)/r}$$

If we expect the result of the pairing,  $A \in \mathbb{F}_{q^k}$ , to be equal to some desired output  $B \in \mathbb{F}_{q^k}$ , we can use the equivalence relation to establish there exists some  $c \in \mathbb{F}_{q^k}$  such that  $A \cdot c^r = B$ .

This can be generalized to  $A \cdot w^{rt} = B$  for  $c = w^t$  in  $A \cdot c^r = B$ .

### 3.3.3 Hasse bound and exponentiation by r

According to Hasse theorem [Sil09],  $q + 1 - r \leq 2\sqrt{q}$  for an elliptic curve over finite field  $\mathbb{F}_q$  and  $r$  the order of the curve.

The exponentiation by  $r$  can be written as exponentiation by  $q + 1 - t$ . Where  $t \leq 2\sqrt{q}$  is much smaller exponentiation in worst case. And the computation of  $c^q$  is efficiently computed by exploiting the Frobenius endomorphisms.

### 3.3.4 Parameters for BN254 curve

Section 4.2 and 4.3 of [NE24] dive into finding optimal parameters for the curve to establish  $A \cdot c^\lambda = B$

For Ethereum's BN254 curve, the optimal parameter is  $\lambda = 6x + 2 + q - q^2 + q^3$ , which decomposes into two efficiently computable components:

1. **Miller loop component**  $(6x + 2)$  — Integrated into existing Miller loop operations
2. **Frobenius component**  $(q - q^2 + q^3)$  — Computed using efficient Frobenius mappings

This eliminates the expensive final exponentiation, replacing it with a few additional Miller loop operations and three Frobenius mappings.

### 3.3.5 Performance

Replacing the final exponentiation by residue witness check saves over 85% of final exponentiation costs for BN254 curve, reducing the constraints required for whole pairing by 32%.

Table 3 summarizes the performance impact of eliminating the final exponentiation step, comparing the original pairing costs with those after applying the residue witness technique.

Table 3: Performance impact of residue witness check [NE24]

| Computation                            | Full Pairing | Witness Check [NE24] | Change        |
|----------------------------------------|--------------|----------------------|---------------|
| <b>Emulated BN254 pairing</b>          |              |                      |               |
| Final Exponentiation                   | 295,823C     | 38,276C              | -87.1%        |
| Total cost                             | 784,050C     | 526,503C             | <b>-32.8%</b> |
| <b>Native BLS12377 [EHG22] pairing</b> |              |                      |               |
| Final Exponentiation                   | 6,130C       | 396C                 | -93.5%        |
| Total cost                             | 14,536C      | 8,802C               | <b>-39.4%</b> |

## 4 Pairing Operations as Polynomials

The techniques discussed in the previous section are outlined in Algorithm 2. This algorithm combines El Housni’s R1CS optimizations [Hou22] with Novakovic and Eagen’s residue witness technique for final exponentiation elimination [NE24].

We now turn our attention to our primary contribution. Building upon Feltroidprime’s direct extension multiplication framework, we present a novel optimization that significantly improves efficiency across all phases of the pairing computation. Our approach consolidates operations across the entire process into unified polynomial verifications. This includes the Miller loop step operations as well as the final exponentiation (via residue witness) verification. Each phase is expressed as a polynomial relationship and verified using the Schwartz-Zippel lemma, substantially reducing constraint count while maintaining cryptographic security.

### 4.1 Strategic Polynomial Segmentation

A critical consideration in our design is managing polynomial degree growth. While consolidating operations into fewer polynomials reduces overhead, we must balance this against polynomial degree constraints to maintain verification efficiency.

#### 4.1.1 Squaring operations and polynomial degree

Squaring operations **double the polynomial degree**. In traditional  $\mathbb{F}_{q^{12}}$  arithmetic, squaring operations are typically cheaper than general multiplications. However, in our polynomial verification framework, squaring the accumulator  $F$  doubles the polynomial degree, making placement critical. When positioned after other operations, squaring becomes the most degree-expensive operation in the sequence.

#### 4.1.2 Natural segmentation points

The optimal points to segment our unified polynomial are immediately *before* squaring the accumulator. By making squaring the first operation in each polynomial segment, we:

1. Minimize the degree growth impact—the squaring happens on a fresh polynomial of degree 11
2. Accumulate all subsequent multiplications at manageable degree increases

---

**Algorithm 2** Optimal Ate pairing with Residue witness check

---

**Input:**  $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}$ Internal  $W, W^2 \in \mathbb{F}_{q^{12}}$  $\triangleright W$  is 27th root of unity**Output:**  $a_{opt}(Q, P)$ 1: Write  $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$  where  $s_i \in \{-1, 0, 1\}$  and  $s_L = 1$ 2:  $c^{-1} \leftarrow 1/c$  $\triangleright$  Inverse Residue witness3:  $f \leftarrow c^{-1}$  $\triangleright$  Initialize with inverse residue witness4:  $T \leftarrow Q$  $\triangleright T_i$  for each pair for multi-pairingEvery operation involving  $T, P$  or  $Q$  is repeated for  $T_i, P_i$  and  $Q_i$  for multi-pairing.5: **for**  $i = L - 2$  to 0 **do**6:    $f \leftarrow f^2$ 7:    $f \leftarrow f \cdot l_{T,T}(P)$  $\triangleright$  Repeat for  $T_i, P_i$ 8:    $T \leftarrow [2]T$  $\triangleright$  Repeat for  $T_i$ 9:   **if**  $s_i = 1$  **then**10:      $f \leftarrow f \cdot c^{-1}$  $\triangleright$  Residue witness11:      $f \leftarrow f \cdot l_{T,Q}(P)$  $\triangleright$  Repeat for  $T_i, Q_i, P_i$ 12:      $T \leftarrow T + Q$  $\triangleright$  Repeat for  $T_i, Q_i$ 13:   **else if**  $s_i = -1$  **then**14:      $f \leftarrow f \cdot c$  $\triangleright$  Residue witness15:      $f \leftarrow f \cdot l_{T,-Q}(P)$  $\triangleright$  Repeat for  $T_i, -Q_i, P_i$ 16:      $T \leftarrow T - Q$  $\triangleright$  Repeat for  $T_i, -Q_i$ 17:   **end if**18: **end for**19:  $Q' \leftarrow \pi_p(Q)$  $\triangleright$  Repeat for  $Q'_i, Q_i$ 20:  $f \leftarrow f \cdot l_{T,Q'}(P)$  $\triangleright$  Repeat for  $Q'_i, T_i, P_i$ 21:  $T \leftarrow T + Q'$  $\triangleright$  Repeat for  $Q'_i, T_i$ 22:  $f \leftarrow f \cdot l_{T,-\pi_p^2(Q)}(P)$  $\triangleright$  Repeat for  $Q_i, T_i, P_i$  $\triangleright$  Skip  $T \leftarrow T + -\pi_p^2(Q)$ 23: **if**  $w = 1$  **then**24:    $f \leftarrow f \cdot W$ 25: **else if**  $w = 2$  **then**26:    $f \leftarrow f \cdot W^2$ 27: **end if**28:  $f \leftarrow f \cdot c^{-q} \cdot c^{q^2} \cdot c^{-q^3}$  $\triangleright$  Uses Frobenius maps, Negative exponents use  $c^{-1}$ 29: **return**  $f$ 

---

### 3. Maintain relatively uniform polynomial degrees across segments

This observation naturally leads to segmenting operations at each Miller loop step boundary, where the accumulator  $F$  is squared. The remaining operations are then segmented to match the degree profile of Miller loop step polynomials, ensuring balanced verification costs across all phases.

## 4.2 Unified Polynomial Framework

Any sequence of field operations in  $\mathbb{F}_{q^{12}}$ <sup>1</sup> can be expressed as a single polynomial equation of the form:

$$A_1(x) \cdot \dots \cdot A_n(x) = Q(x) \cdot P_{12}(x) + R(x) \text{ where } A_1, \dots, A_n, R \in \mathbb{F}_{q^{12}}$$

This unified framework allows us to batch verify entire computational phases rather than individual operations, achieving dramatic improvements in both constraint counts and commitment overhead across the complete pairing computation.

## 4.3 Comparative Analysis

The Miller loop is comprised of steps for each individual bit, which in NAF (non-adjacent form) [HMOV04], may be +1, -1 or 0. Instead of performing a polynomial commitment for each individual extension field arithmetic operation separately, we propose the whole bit operation be viewed as one polynomial. And commitment be made to that polynomial.

To get a closer look into effectiveness of our scheme, we will make a real world application comparison for verification of three-pairing Miller loop which is common for verification of Groth16 [Gro16].

### 4.3.1 Previous scheme [Fel23]

**Adapting faster extension field multiplication** [Fel23] idea to line 6 and 7 of Miller loop algorithm 2 for a 3-pair Miller loop, we have following polynomial relationships to assert.

$$\begin{aligned} F(x) \cdot F(x) &\stackrel{?}{=} Q_1(x) \cdot P_{12}(x) + T_1(x) \\ T_1(x) \cdot L_1(x) &\stackrel{?}{=} Q_2(x) \cdot P_{12}(x) + T_2(x) \\ T_2(x) \cdot L_2(x) &\stackrel{?}{=} Q_3(x) \cdot P_{12}(x) + T_3(x) \\ T_3(x) \cdot L_3(x) &\stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x) \end{aligned}$$

$F, T_1, T_2, T_3, R$  cost 12C each for evaluation, 4C for  $\mathbb{F}_q$  multiplication, and  $L_1, L_2, L_3$  require 4C each for evaluation. All the remainder hints need to be provided for commitment, so this is 12 calldata commitment for  $R$  and intermediate remainder polynomials  $T_1, T_2, T_3$ . **Costs sum up to 76C** (  $5 * 12C + 3 * 4C + 4C$  ) **and 48 commitments** in  $\mathbb{F}_q$  (  $4 * 12$  ). Resulting remainder polynomial  $R \in \mathbb{F}_{q^{12}}$  is the result of the Miller loop step.

---

<sup>1</sup>Elements need not have all 12 non-zero coefficients, but should align with the  $\mathbb{F}_{q^{12}}$  representation (e.g., sparse line functions).

### 4.3.2 Our scheme

**In our approach**, instead of working with individual arithmetic operations, we treat the entire bit operation as a single polynomial. In our scheme line 6 and 7 of 3-pair Miller loop (Algorithm 2) looks like this.

$$F(x) \cdot F(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) \stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)$$

Similar to the previous approach,  $F$  and  $R$  cost 12C each for evaluation, 4C for  $\mathbb{F}_q$  multiplication, and  $L_1, L_2, L_3$  require 4C each for evaluation, but we eliminate intermediate ( $T_*$  polynomials above) commitments and evaluations. Only the remainder hint needs commitment, so this is 12 calldata commitment for  $R$ . **Costs sum up to 40C** ( $2 * 12C + 3 * 4C + 4C$ ) **and 12  $\mathbb{F}_q$  commitments.**

This results in **47% fewer constraints** and a whopping **75% savings in the commitments**. When implemented on a blockchains for verification of ZK proofs, commitment savings become far more critical because of limitations and costs associated with calldata needed for hints.

## 4.4 Bit-Type Polynomials

Our implementation uses different polynomials for different bits in the Miller loop. Let's walk through a 3-pairing setup which Inversions used widely for Groth16 verification. We will describe the polynomials for zero bits, non-zero bits and correction step.

### 4.4.1 Zero Bit Equation

For zero bit operations (line 6 and 7 with three  $l_{T,T}(P)$  lines in Algorithm 2), the Miller loop just performs doubling. So for each pair we have  $\text{Sparse}_{01379}$  lines. The verification equation becomes:

$$F(x) \cdot F(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) = R(x) + Q(x) \cdot P_{12}(x)$$

| Polynomial                                | Description                                      | Degree             | Coefficients      |
|-------------------------------------------|--------------------------------------------------|--------------------|-------------------|
| $F \in \mathbb{F}_{q^{12}}$               | Miller loop accumulator                          | $11 \times 2 = 22$ | 12                |
| $L_1, L_2, L_3 \in \text{Sparse}_{01379}$ | The line functions                               | $9 \times 3 = 27$  | $4 \times 3 = 12$ |
| $R \in \mathbb{F}_{q^{12}}$               | Remainder, The result                            | 11                 | 12                |
| <b>Total multiplications</b>              |                                                  |                    | <b>36</b>         |
| Q Amortized                               | $\text{LHS}_{\text{Degree}} - R_{\text{Degree}}$ | 38                 | 39                |

Table 4: Zero bit operation polynomial components

### 4.4.2 Non-Zero Bit Equation

For non-zero bit operations (line 6, 7 and lines 9 to 17 except 12 and 16, with three pairs of lines in Algorithm 2), the Miller loop performs both doubling and addition steps. So for each pair we have two  $\text{Sparse}_{01379}$  lines. The verification equation becomes:

$$F(x) \cdot F(x) \cdot W(x) \cdot L_{d1}(x) \cdot L_{d2}(x) \cdot L_{d3}(x) \cdot L_{a1}(x) \cdot L_{a2}(x) \cdot L_{a3}(x) = R(x) + Q(x) \cdot P_{12}(x)$$

| Polynomial                                         | Description                                      | Degree             | Coefficients      |
|----------------------------------------------------|--------------------------------------------------|--------------------|-------------------|
| $F \in \mathbb{F}_{q^{12}}$                        | Miller loop accumulator                          | $11 \times 2 = 22$ | 12                |
| $L_{d1}, L_{d2}, L_{d3} \in \text{Sparse}_{01379}$ | Doubling line functions                          | $9 \times 3 = 27$  | $4 \times 3 = 12$ |
| $L_{a1}, L_{a2}, L_{a3} \in \text{Sparse}_{01379}$ | Addition line functions                          | $9 \times 3 = 27$  | $4 \times 3 = 12$ |
| $W$ or $W^{-1} \in \mathbb{F}_{q^{12}}$            | Residue witness                                  | 11                 | 12                |
| $R \in \mathbb{F}_{q^{12}}$                        | Remainder, The result                            | 11                 | 12                |
| <b>Total multiplications</b>                       |                                                  |                    | <b>60</b>         |
| Q Amortized                                        | $\text{LHS}_{\text{Degree}} - R_{\text{Degree}}$ | 76                 | 77                |

Table 5: Non-zero bit operation polynomial components

The residue witness  $W$  encapsulates the equivalence class relationship established by Novakovic and Eagen’s final exponentiation elimination. The inverse of  $W$  is used for negative bits, this just changes the coefficients the equation remains the same.

#### 4.4.3 Correction Step Equation

## 5 Interactive Oracle Proof

### 5.1 Polynomial Accumulation and implementation

The commitment can include all the required  $R$ s for each equation we want to prove. We arrive at the input elements through computation in the circuit, the commitments array is popped on reaching the point in the computation. We obtain an RLC (Random Linear Combination) of all LHS evaluations and precompute  $P_{12}$ . For example, for zero bits we go for,

$$F(x) \cdot F(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) - R(x) = Q(x) \cdot P_{12}(x)$$

or,

$$\text{LHS} = F(x) \cdot F(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) - R(x)$$

We then prepare an RLC of each equation to prove, for  $n$  equations we have,

$$\sum_{i=1}^n c_i \cdot \text{LHS}_i = \sum_{i=1}^n c_i \cdot Q_i(x) \cdot P_{12}(x)$$

Along with the commitments to each  $R$  we expect the prover to provide the correct RLC polynomial for  $\sum_{i=1}^n c_i \cdot Q_i$ . Here,  $x$  and all  $c_i$  are obtained from Fiat Shamir heuristics over all  $R_i$  commitments. This prevents the prover from guessing the randomness before commitment. But allows her to aggregate RLC for all quotients.

#### 5.1.1 Prover

Prover performs the pairing verification herself, and computes and stores all  $R_i$  and  $Q_i$ . After the pairing computation, she uses same hashing method as the verifier to obtain all  $c_i$ . She then computes the RLC of all  $Q_i$  and sends it along with all  $R_i$  commitments.

### 5.1.2 Verifier

Verifier receives all the remainders,  $R_i$ , and RLC of quotients -  $\sum_{i=1}^n c_i \cdot Q_i$ . The verifier then prepares all  $c_i$  solely from the remainder coefficients. For the evaluation point  $x$ , she includes all coefficients from the quotient RLC in the Fiat-Shamir hash as well. This guarantees soundness in Fiat Shamir. Then she uses generated  $c_i$  for aggregating  $LHS_i$  evaluated at  $x$ . In the end, she subtracts the evaluation of quotients RLC at  $x$  times  $P_{12}(x)$  from the aggregated LHS evaluation. If the result is zero, according to Schwartz–Zippel lemma, all the provided  $R_i$  are correct with a very high<sup>2</sup> probability.

## 5.2 Performance

Batching operations in Miller loop steps yielded substantial performance improvements. Processing all operations within a Miller loop step as a single polynomial and performing Schwartz-Zippel verification of polynomial multiplications significantly reduced the overall computational requirements of the pairing verification process. Combined with residue witness technique for final exponentiation elimination [NE24], these optimizations enabled processing of ZK proofs on Starknet even before the modulo builtin [Sta] became available, which later greatly improved finite field arithmetic operations. These techniques were subsequently integrated into Garaga, which remains the most efficient pairing implementation by operation count on Starknet to date.

## 5.3 [wip] Protocol Overview

## 5.4 [wip] Single-Round Protocol

### 5.4.1 [wip] Protocol description (Algorithm)

### 5.4.2 [wip] Complexity analysis

## 5.5 Two-Round Protocol with RLC Batching

### 5.5.1 [wip] Protocol description (Algorithm)

### 5.5.2 [wip] Complexity analysis

### 5.5.3 [wip] Comparison with single-round

## 5.6 Three-Round Protocol with Native Field Optimization

### 5.6.1 [wip] Protocol description (Algorithm)

### 5.6.2 [wip] Complexity analysis

### 5.6.3 [wip] Architecture-specific benefits

## References

- [ABLR14] Diego F. Aranha, Paulo S. L. M. Barreto, Patrick Longa, and Jefferson E. Ricardini. The realm of the pairings. In Tanja Lange, Kristin Lauter, and

---

<sup>2</sup> $Pr[P(r_0, r_1, r_2, \dots, r_d) = 0] \leq \frac{d}{|\mathbb{F}|}$

- Petr Lisoněk, editors, *Selected Areas in Cryptography – SAC 2013*, pages 3–25, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.
- [BDM<sup>+</sup>10] Jean-Luc Beuchat, Jorge Enrique González Díaz, Shigeo Mitsunari, Eiji Okamoto, Francisco Rodríguez-Henríquez, and Tadanori Teruya. High-speed software implementation of the optimal ate pairing over barreto-naehrig curves. Cryptology ePrint Archive, Paper 2010/354, 2010.
  - [BGW<sup>+</sup>22] Dan Boneh, Sergey Gorbunov, Riad S. Wahby, Hoeteck Wee, Christopher A. Wood, and Zhenfei Zhang. BLS Signatures. Internet-draft, Internet Engineering Task Force, June 2022. Work in Progress.
  - [Con] Consensys. gnark: A fast zk-snark library. <https://github.com/Consensys/gnark>.
  - [Cos19] Craig Costello. Pairings for beginners. Tutorial, 2019. Available at <https://www.craigcostello.com.au/tutorials>.
  - [DL78] Richard A. Demillo and Richard J. Lipton. A probabilistic remark on algebraic program testing. *Information Processing Letters*, 7(4):193–195, 1978.
  - [EHG22] Youssef El Housni and Aurore Guillevic. Families of snark-friendly 2-chains of elliptic curves. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022*, pages 367–396, Cham, 2022. Springer International Publishing.
  - [Fel] Feltroidprime. Bijection between  $\text{fp}_{12}/\text{fp}_6/\text{fp}_2$  tower representation and direct  $\text{fp}_{12}/\text{fp}$  extension for  $\text{bn}_{254}$ . <https://gist.github.com/feltroidprime/bd31ab8e0cbc0bf8cd952c8b8ed55bf5>.
  - [Fel23] Feltroidprime. Faster extension field multiplications for emulated pairing circuits. <https://hackmd.io/@feltroidprime/B1eyHHXNT>, 2023. HackMD.
  - [GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct nizks without pcps. In *Advances in Cryptology - EUROCRYPT 2013*, pages 626–645. Springer, 2013. Foundational QAP framework for R1CS.
  - [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. <https://eprint.iacr.org/2016/260>, 2016.
  - [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. <https://eprint.iacr.org/2019/953>, 2019. Cryptology ePrint Archive, Paper 2019/953.
  - [HMOV04] Darrel Hankerson, Alfred J. Menezes, and Scott Vanstone. *Guide to Elliptic Curve Cryptography*. Springer-Verlag, 2004. Non-adjacent form representation.
  - [Hou22] Youssef El Housni. Pairings in rank-1 constraint systems. <https://eprint.iacr.org/2022/1162>, 2022.



- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010*, pages 177–194, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [NE24] Andrija Novakovic and Liam Eagen. On proving pairings. <https://eprint.iacr.org/2024/640>, 2024. Cryptology ePrint Archive, Paper 2024/640.
- [Sch79] Jacob T. Schwartz. Probabilistic algorithms for verification of polynomial identities. In Edward W. Ng, editor, *Symbolic and Algebraic Computation*, pages 200–215, Berlin, Heidelberg, 1979. Springer Berlin Heidelberg.
- [Sch80] J. T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, October 1980.
- [Sil09] Joseph H. Silverman. *The Arithmetic of Elliptic Curves*. Graduate Texts in Mathematics. Springer, New York, NY, 2 edition, 2009. 14 b/w illustrations.
- [Sta] Starknet. The mod builtin. <https://docs.starknet.io/build/corelib/core-circuit>. Cairo Book - Chapter 2.4.2.8.
- [Sta22] Starkware. Cairo: A turing-complete stark-friendly cpu architecture. <https://www.cairo-lang.org/>, 2022.
- [Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In Edward W. Ng, editor, *Symbolic and Algebraic Computation*, pages 216–226, Berlin, Heidelberg, 1979. Springer Berlin Heidelberg.